

Agglomerative Clustering

Bottom-up hierarchical clustering: from single points to a complete tree of structure

Concepts · Algorithm · Linkage & Math · Complexity · Python · Applications



Clustering: Finding Structure

Grouping unlabeled data by similarity

Clustering is unsupervised learning: it groups unlabeled data so that items in the same cluster are more similar to each other than to items in other clusters. "Similar" is defined by a distance metric.



Partitional (flat)

Splits the data into k non-overlapping groups in one shot; k must be chosen up front.

K-Means · K-Medoids · Gaussian Mixtures



Hierarchical (tree)

Builds nested clusters as a tree, so every level from 1 to n clusters is available.

Agglomerative: bottom-up (merge)
Divisive: top-down (split)



Points that share a colour are closer to each other than to points in other groups.

What Is Agglomerative Clustering?

A bottom-up hierarchical method that merges the closest clusters, step by step

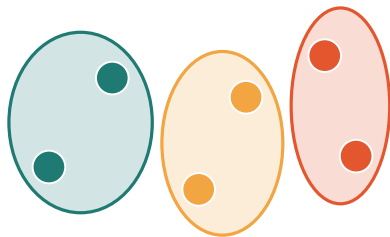
DEFINITION

Every data point starts as its own cluster. At each step the two closest clusters are merged, until a single cluster containing all data remains. The full merge history forms a tree called a dendrogram.

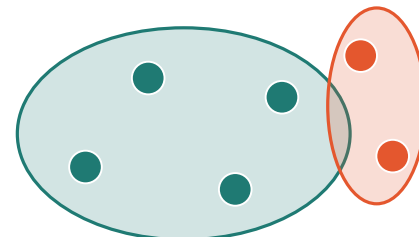
Start: 6 clusters



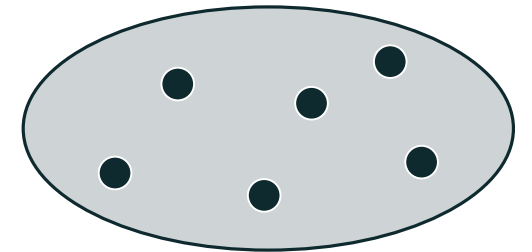
After 3 merges: 3 clusters



After 4 merges: 2 clusters



After 5 merges: 1 cluster



Why Use It?

What problem does it solve compared with flat clustering?



No need to choose k in advance

The whole hierarchy is built once; cut it at any level afterwards.



Reveals nested structure

Clusters within clusters: taxonomies, gene families, product categories.



Flexible with distances

Works with any dissimilarity: Euclidean, cosine, correlation, Jaccard, custom.



Deterministic and interpretable

No random initialisation; the dendrogram shows every single merge.

ONE TREE, ANY k

$n \rightarrow 1$

Clusters shrink from n singletons to a single root, and every level in between is a valid clustering.

K-Means: pick k first, hope for the best.

Agglomerative: see the whole picture, decide later.

The Algorithm, Step by Step

$n - 1$ merges turn n singletons into one cluster

1

Initialise

Treat each of the n data points as its own cluster.

2

Compute

Build the $n \times n$ matrix of pairwise distances between points.

3

Merge

Find the closest pair of clusters and merge them into one.

4

Update

Recompute distances from the new cluster to all others using the linkage rule.

5

Repeat

Continue until one cluster remains; record each merge height.

```
clusters = { {x1}, {x2}, ..., {xn} }
while len(clusters) > 1:
    (Ci, Cj) = pair with smallest linkage distance d(Ci, Cj)
    Cnew = merge(Ci, Cj)      # record height = d(Ci, Cj)
    update d(Cnew, Ck) for every other cluster Ck
```

Result

A merge history of $n - 1$ rows (cluster i , cluster j , height, size), from which the dendrogram is drawn.

Worked Example (Single Linkage)

Five points on a line: A = 1, B = 2, C = 6, D = 8, E = 15

Pairwise distance matrix $d = |x - y|$

	A	B	C	D	E
A	0	1	5	7	14
B	1	0	4	6	13
C	5	4	0	2	9
D	7	6	2	0	7
E	14	13	9	7	0

Amber = closest pairs, merged first (heights 1 and 2).

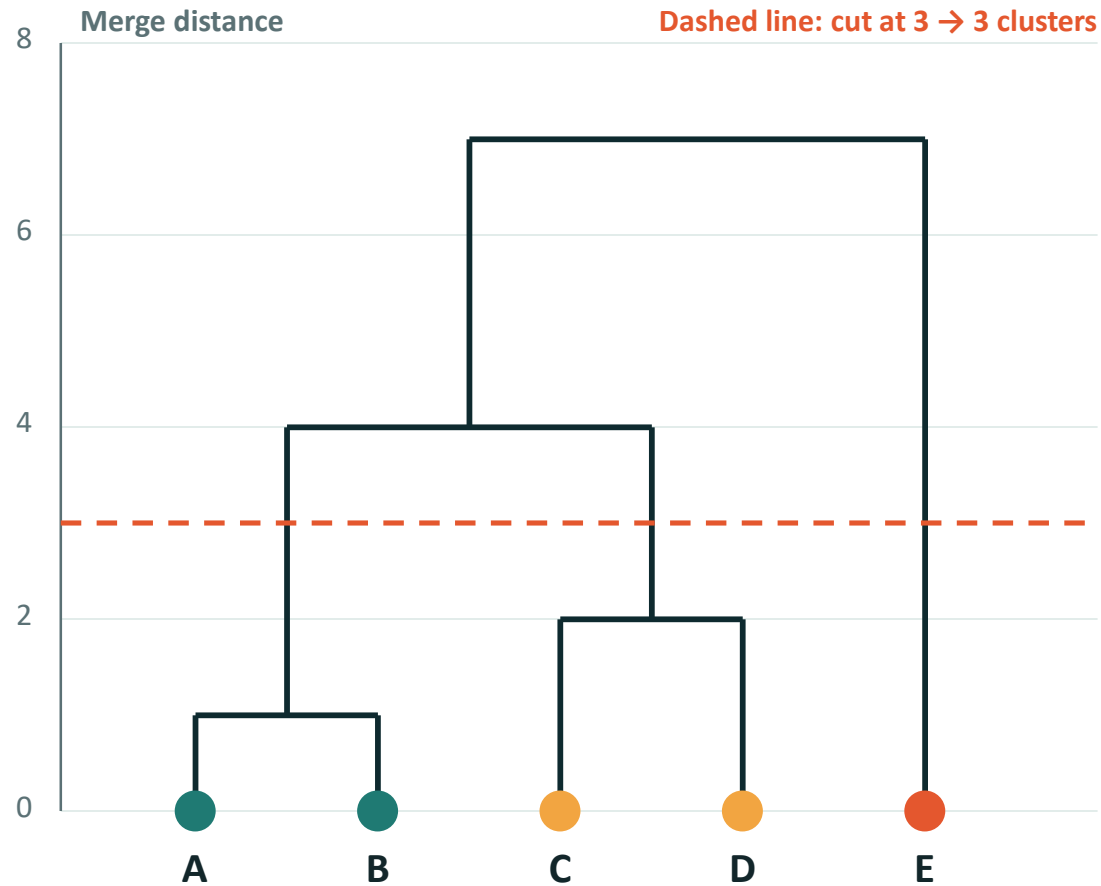
Merge history

Step	Merge	Height	Why
1	A + B	1	Smallest entry, $d(A,B) = 1$
2	C + D	2	Next smallest, $d(C,D) = 2$
3	$\{A,B\} + \{C,D\}$	4	min cross-pair = $d(B,C) = 4$
4	$\{A,B,C,D\} + E$	7	min cross-pair = $d(D,E) = 7$

Single-linkage update: $d(\{A,B\}, C) = \min(d(A,C), d(B,C)) = \min(5, 4) = 4$. The merge heights 1, 2, 4, 7 become the dendrogram on the next slide.

Reading the Dendrogram

The tree from the worked example (single linkage)



Leaves = data points

Each leaf at the bottom is one original observation.



U-shaped joins = merges

The height of each join is the distance at which two clusters merged.



A horizontal cut = a clustering

Every vertical line the cut crosses is one cluster: cut at 3 gives {A,B}, {C,D}, {E}.



Linkage changes the tree

Complete linkage on the same data merges at 1, 2, 7, 14: a taller, different tree.

Distance Metrics

Step 0: define how far apart two points are (Ward and centroid linkage require Euclidean)



Euclidean (L2)

$$d(x,y) = \sqrt{(\sum (x_i - y_i)^2)}$$

Default for continuous, scaled features; required by Ward.



Manhattan (L1)

$$d(x,y) = \sum |x_i - y_i|$$

Grid-like paths; less dominated by one large difference.



Minkowski (Lp)

$$d(x,y) = (\sum |x_i - y_i|^p)^{1/p}$$

Generalises L1 ($p = 1$), L2 ($p = 2$) and Chebyshev ($p \rightarrow \infty$).



Cosine

$$d(x,y) = 1 - (x \cdot y) / (\|x\| \|y\|)$$

Compares direction, not magnitude: text, embeddings.



Correlation

$$d(x,y) = 1 - r(x,y)$$

Compares profile shape: gene expression, time series.



Jaccard / Hamming

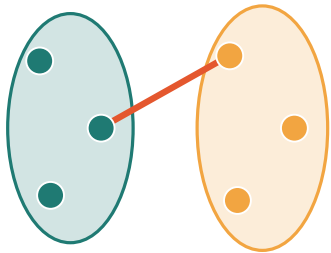
$$d(A,B) = 1 - |A \cap B| / |A \cup B|$$

Sets and binary or categorical features. Hamming = share of positions that differ.

Linkage Criteria

How far apart are two clusters, not just two points?

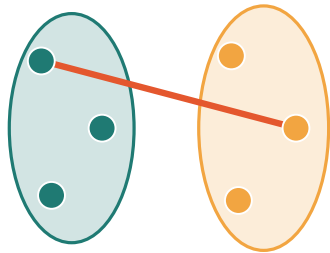
Single



$$d = \min d(a,b)$$

Uses the closest pair. Follows elongated shapes but chains through noise.

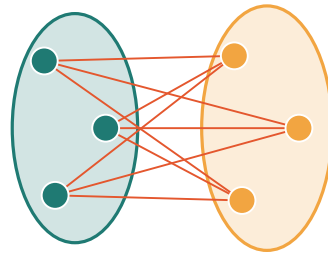
Complete



$$d = \max d(a,b)$$

Uses the farthest pair. Compact clusters; sensitive to outliers.

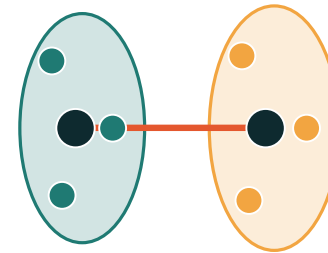
Average



$$d = \text{mean of all } d(a,b)$$

Averages every cross pair (UPGMA). A robust middle ground.

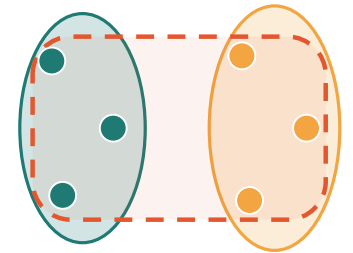
Centroid



$$d = \|\mu_A - \mu_B\|$$

Distance between centres. Simple, but can produce inversions.

Ward



$$\Delta = w \cdot \|\mu_A - \mu_B\|^2$$
$$w = |A||B|/(|A|+|B|)$$

Minimises the growth in within-cluster variance. Compact, balanced.

Choosing a Linkage

Behaviour of each criterion at a glance

Linkage	Favours shapes	Outliers and noise	Distance	Best when
Single	Elongated, chain-like	High: chains through noise	Any	Non-globular shapes in clean data
Complete	Compact, similar diameter	High: one outlier stretches the max	Any	Tight, well-separated groups
Average	Moderately compact	Moderate	Any	General purpose; non-Euclidean distances
Centroid	Compact	Moderate	Euclidean	Quick exploration (watch for inversions)
Ward	Compact, similar size	Moderate	Euclidean	Default for scaled numeric data



Rule of thumb: start with Ward on standardised numeric data; try average linkage with cosine or other custom metrics; use single linkage only when chain-like clusters are expected.

The Math: Lance–Williams Update

One formula computes every linkage, without revisiting the raw data

$$d(k, i \cup j) = \alpha_i \cdot d(k, i) + \alpha_j \cdot d(k, j) + \beta \cdot d(i, j) + \gamma \cdot |d(k, i) - d(k, j)|$$

After merging clusters i and j , the distance to any other cluster k is derived from stored distances.

Method	α_i	α_j	β	γ
Single	1/2	1/2	0	-1/2
Complete	1/2	1/2	0	+1/2
Average	$n_i/(n_i+n_j)$	$n_j/(n_i+n_j)$	0	0
Centroid*	$n_i/(n_i+n_j)$	$n_j/(n_i+n_j)$	$-n_i n_j/(n_i+n_j)^2$	0
Ward*	$(n_i+n_k)/(n_i+n_j+n_k)$	$(n_j+n_k)/(n_i+n_j+n_k)$	$-n_k/(n_i+n_j+n_k)$	0

* Exact for squared Euclidean distances. n_i , n_j , n_k are cluster sizes.

SANITY CHECK

Single linkage: $\frac{1}{2}a + \frac{1}{2}b - \frac{1}{2}|a - b| = \min(a, b)$

Example: $d(\{A, B\}, C)$ with $a = 5$, $b = 4$ gives $2.5 + 2 - 0.5 = 4$, matching the worked example.

Ward cost: the increase in total within-cluster sum of squares caused by the merge.

Cutting the Dendrogram

How many clusters should we keep?



Longest vertical gap

Cut where merge heights jump the most (the visual "elbow").



Distance threshold

Cut at a fixed height: `distance_threshold` in scikit-learn, or `fcluster` with `criterion="distance"`.



Silhouette score

Try $k = 2 \dots K$ and keep the k with the highest mean silhouette $s = (b - a) / \max(a, b)$.



Cophenetic correlation

Correlates dendrogram distances with original distances; judges how faithful a linkage is.

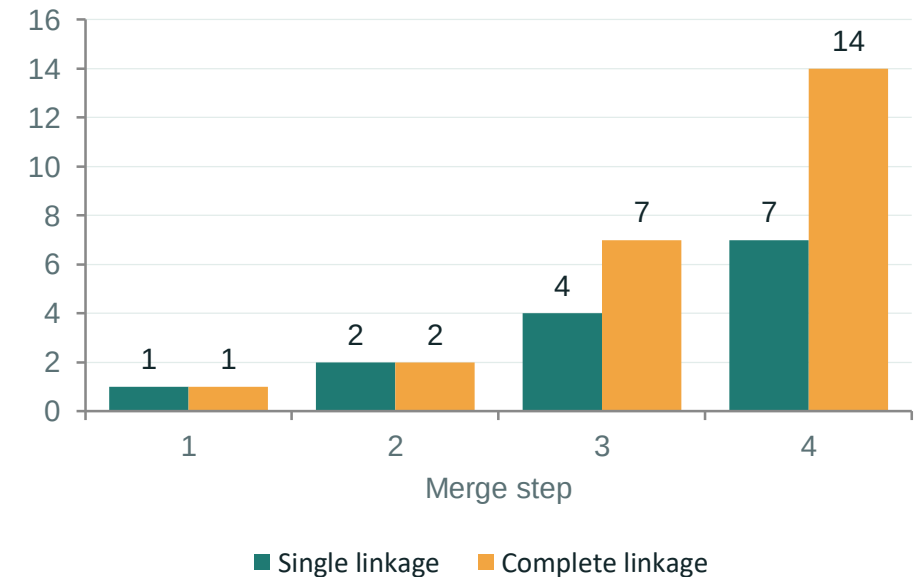


Inconsistency coefficient

Flags merges much taller than nearby merges (SciPy `inconsistent` / `fcluster`).

Merge height at each step

Worked-example data, single vs complete linkage



A large jump means two dissimilar clusters are being joined, so cut just below the jump.

Complexity & Scalability

The distance matrix is the bottleneck

Approach	Time	Memory
Naive: rescan the matrix at every step	$O(n^3)$	$O(n^2)$
Priority queue / heap per cluster	$O(n^2 \log n)$	$O(n^2)$
Nearest-neighbour chain*	$O(n^2)$	$O(n^2)$
SLINK (single linkage only)	$O(n^2)$	$O(n)$

* Valid for "reducible" linkages: single, complete, average, Ward (not centroid or median).

SCALING TIPS

- Subsample, or pre-cluster with BIRCH / MiniBatch K-Means
- Add a k-NN connectivity graph to restrict merges
- Reduce dimensions first (PCA) or try HDBSCAN

Memory for the distance matrix (float64)

4 MB

$n = 1,000$

400 MB

$n = 10,000$

10 GB

$n = 50,000$

40 GB

$n = 100,000$

Condensed matrix: $n(n - 1)/2$ entries $\times$ 8 bytes. Quadrupling n multiplies memory by ~ 16 .

Strengths & Limitations

Honest trade-offs



Advantages

- No need to specify k in advance
- Full hierarchy: a multi-resolution view of the data
- Deterministic: no random initialisation
- Works with any distance or similarity
- Dendrogram is visual and interpretable
- Can capture non-spherical shapes (single linkage)



Disadvantages

- Costly: $O(n^2)$ memory, $O(n^2)$ to $O(n^3)$ time
- Greedy: a merge can never be undone
- Sensitive to noise, outliers and linkage choice
- Chaining (single) and size bias (complete, Ward)
- Choosing where to cut is subjective
- Distances lose meaning in very high dimensions

Alternatives at a Glance

Where agglomerative clustering fits among clustering methods

Algorithm	Needs k?	Cluster shapes	Scalability	Best for
Agglomerative	No (cut the tree)	Depends on linkage	Low: $O(n^2)$ memory	Small to medium data where a hierarchy is wanted
Divisive (DIANA)	No (cut the tree)	Depends on split rule	Low	Top-level splits matter most; can stop early
K-Means	Yes	Spherical, convex	Excellent	Large data with compact, similar-size clusters
DBSCAN	No (eps, minPts)	Arbitrary	Good	Noisy data, arbitrary shapes, uniform density
HDBSCAN	No (min size)	Arbitrary, varying density	Good	Varying density; gives a cluster hierarchy too
Gaussian Mixture	Yes	Elliptical	Good	Soft (probabilistic) cluster assignments

Highlighted row: the topic of this deck.

Where It's Used

Anywhere nested groups matter



Bioinformatics

Gene-expression heatmaps and phylogenetic trees (UPGMA is agglomerative).



Customer segmentation

Nested segments: broad personas that split into finer micro-segments.



Document clustering

Topic taxonomies from text using cosine distance on embeddings.



Image segmentation

Merge similar pixels or regions into superpixels and objects.



Anomaly detection

Points that join the tree very late, at large heights, are outlier candidates.



Community detection

Greedy modularity merging builds a hierarchy of network communities.

Best Practices & Optimal Choice

What to use, and how to get reliable results

RECOMMENDED DEFAULT

Ward + Euclidean

on standardised numeric features

It minimises within-cluster variance, giving compact, balanced clusters like K-Means, but with a full hierarchy.

Switch away when

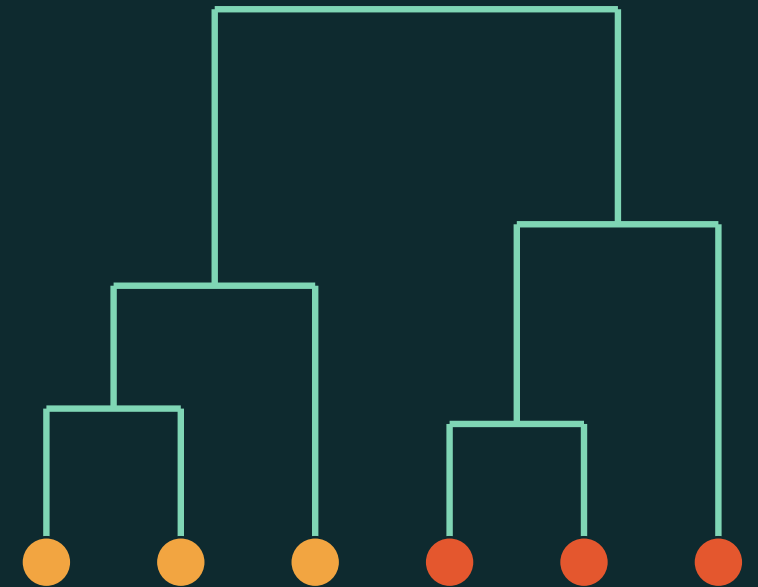
- $n > \sim 20\text{--}50\text{k}$: pre-cluster first
- Noise or varying density: HDBSCAN
- Streaming data: MiniBatch K-Means

Best-practice checklist

- 1 **Standardise features.** Distances are scale-sensitive; use z-scores (or cosine for text).
- 2 **Reduce dimensions.** Apply PCA when features are many, since distances lose contrast.
- 3 **Match linkage to shape.** Ward/complete for compact, single for chains, average as a compromise.
- 4 **Validate.** Check silhouette, cophenetic correlation and the dendrogram itself.
- 5 **Inspect outliers first.** A few extreme points can distort complete and Ward merges.
- 6 **Scale up smartly.** Use a connectivity graph or cluster centroids from a fast pre-clustering.

Key Takeaways

- 1 Bottom-up.** Start with n singletons and merge the closest pair $n - 1$ times.
- 2 Linkage defines "closest".** Single, complete, average, centroid or Ward, all via Lance–Williams.
- 3 The dendrogram is the result.** Cut it anywhere to get any number of clusters.
- 4 It is expensive.** $O(n^2)$ memory makes it best for small to medium datasets.
- 5 Sensible default.** Standardise, use Ward + Euclidean, validate with silhouette.



Test Your Understanding

Three questions to check what you have learned

Q1



CALCULATE

Points on a line: $A = 1$, $B = 2$, $C = 6$, $D = 8$, $E = 15$. At what distance does the final merge happen under (a) single linkage and (b) complete linkage?

Q2



EXPLAIN

What is the chaining effect? Which linkage causes it, and which linkage would you pick for compact, similar-sized clusters? Why?

Q3



APPLY

You have 200,000 customer records. Why is plain agglomerative clustering a poor fit, and what would you do instead?